Near Field Diffraction and Holography

Final Report

May 1, 2020

Submitted to: Dr. Gregory Lafyatis
614-292-2282
lafyatis@physics.osu.edu

Submitted by: Ayub Sherif
614-446-7260
sherif.8@buckeyemail.osu.edu

Chris Esparza
614-531-9615
esparza.12@buckeyemail.osu.edu

Zachary Cotman
614-546-8126
cotman.6@buckeyemail.osu.edu

Table of Contents

Near Field Diffraction and Digital Holography **3**

Project Status **5**

Introduction 5

Optical Setup 6

Near Field Work 8

Single Slit Fresnel Diffraction 8

Talbot Carpet 11

Grating Fabrication 12

Talbot Carpet Image Processing 15

Holography 19

Introduction to Holography 19

Holographic Diffraction Patterns on GPU 20

Holographic Diffraction in MATLAB: Fresnel Transform 24

Potential Next Steps **26**

Near Field 26

Camera 26

Absolute Image Distance 26

Talbot Images 27

Laser Characterization 27

Holography 27

Kinect 27

Cuda 28

MATLAB Hologram Calculations 29

Appendix **30**

Appendix A: Lightcrafter EVM and DMD(DLP 3000) 30

Interfacing 30

Specifications 30

Usable Inputs 31

Appendix B: Matlab Holographic Calculation 32

Appendix C: CUDA and OpenGL 33

Near Field Diffraction and Digital Holography

Evan Cornuelle, Zachary Cotman, Christ Esparza, Larry King, Chris West

Diffraction is a manifestation of interference of waves. One of the earliest intuitive understandings of this was made by Christiaan Huygens in the late 17th century. He suggested that light could be thought of as a wave phenomenon and to describe the forward motion of the wave front, any wave front could be considered a line of tiny spherical emitters. It took over a century for the wave model of light to be proven correct, and there are now much more rigorous descriptions of the propagation of light as a continuum, but Huygens' theory still remains exceedingly useful, especially in computation where continuums are, by necessity, discretized. It is from this basis that the following project is built. [1][2][3]

The primary motivation for the near-field project is the creation of an undergraduate near-field optics lab, where near field diffraction patterns can be captured experimentally with the ultimate goal being the realization of talbot images. In attempting to achieve a reasonable undergraduate lab, the project also steers towards fewer and less-expensive optics. The advent of cameras with very small pixel sizes making this feasible. Whereas, past experimental Talbot realizations relied on expensive imaging optics between the grating and the camera, newer, inexpensive cameras with micron-sized pixels can be placed directly in front of the grating and capture the diffraction patterns. Figure 1 displays a qualitative comparison between some experimental near-field diffraction patterns and simulated diffraction patterns. Figure 2 shows a comparison between experimental and simulated Talbot carpets. Details about the optical setup and grating fabrication can be found in the attached report.

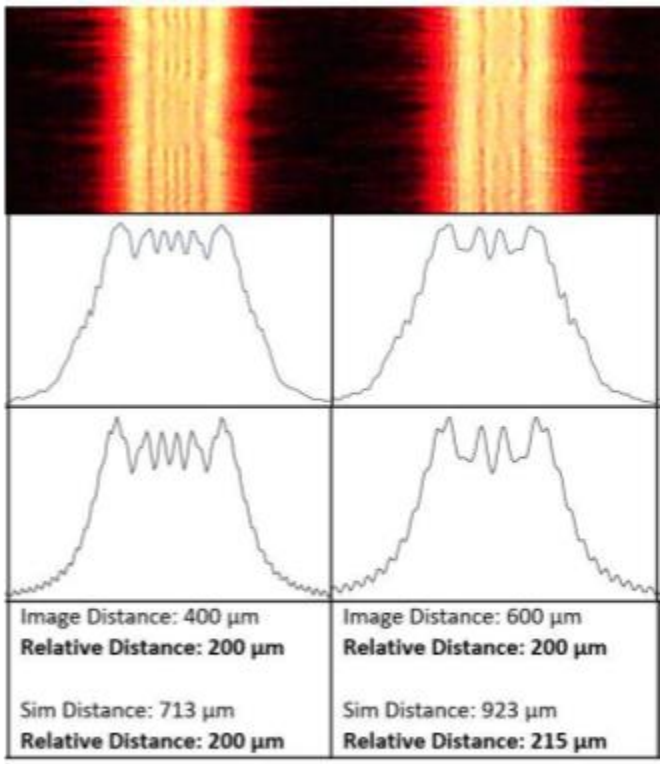

Figure 1: Near field single slit diffraction, (top) experimental images, (middle) experimental intensity plot, (bottom) simulated intensity plot

The holography part of the project is motivated by the proliferation of holograms such as that of Princes Leia, and the advent of digital displays that have the capability of creating moving holograms. The DLP 3000, a digital micromirror device (DMD), used in this project is one such display that utilizes an array of micromirrors to reflect light. Combining this display with the

computational speed that GPU's can now provide, we have achieved real-time holographic image display. Figure 3 displays some examples of holographic diffraction patterns and images.

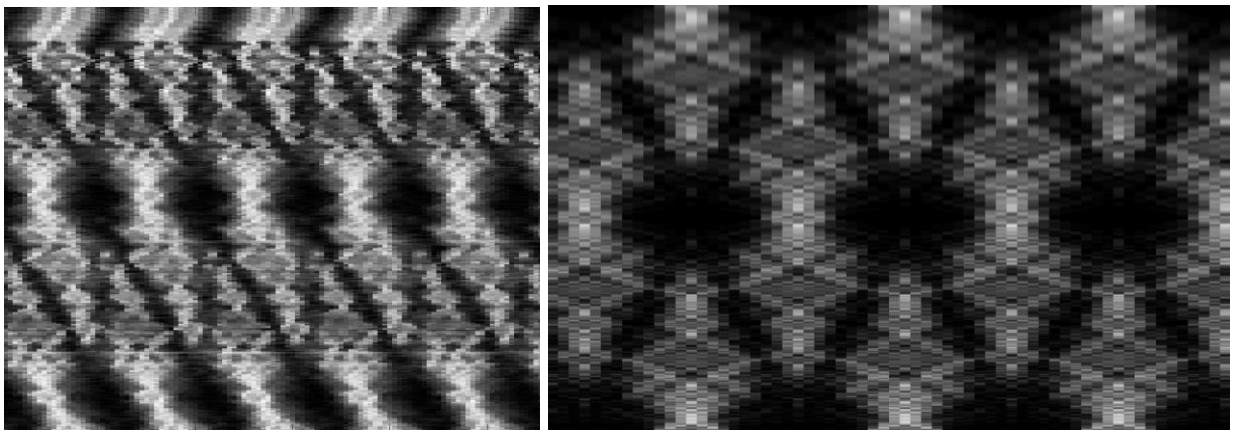

Figure 2: (left) Experimental Talbot carpet. (right) Simulated Talbot carpet

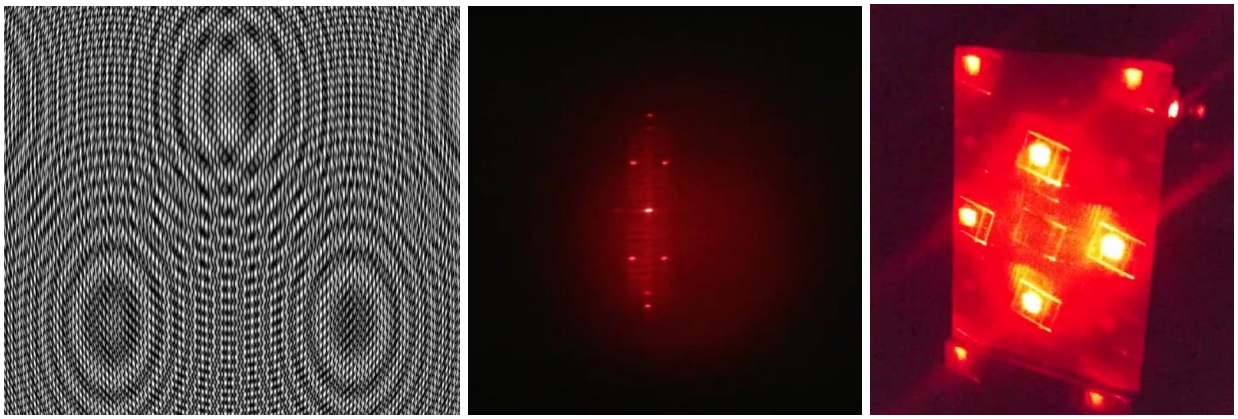

Figure 3: (left) Calculated diffraction pattern, (center) 3-Point hologram as viewed by a camera, (right) Rotating cube hologram projected onto a screen.

Results obtained thus far are very promising, and if they can be built off of in the future, fun holograms and an educational near field lab can be produced. It is also notable that much of this work shares its fundamental nature with signal processing. Knowledge of which is extremely applicable in science and engineering fields today.

References

1. Hecht, E., Optics, 4th Ed., 2002
2. Goodman, J.W. Introduction to Fourier Optics, 1968
3. Schmidt, Jason D., Numerical Simulation of Optical Wave Propagation, With examples in MATLAB

Project Status

Introduction

The following material is intended for use by the next engineering capstone group to work on this project.

The primary topics this project involves are:

- Propagation of light
- Physical optics
- Fourier transforms and signal processing
- Computation

Having a basic understanding of the mathematics involved in the propagation of light is a necessity for executing this project well. This includes understanding the discretized summation that is the mathematical representation of Huygens' principle as well as its integral form. It is also necessary to understand how the far-field solutions, called Fraunhofer diffraction, and one form of the near-field solutions, called Fresnel diffraction, are arrived at using first, and then second-order taylor series expansions. For in-depth discussions of these topics see *Optics 4th Ed* by Eugene Hecht, and *Numerical Simulation of Wave Propagation, with Examples in MATLAB* by Jason Schmidt, which can be found in the repository. Another great resource is *Introduction to Fourier Optics* by J. W. Goodman.

Experience with physical optics is highly beneficial. There are great resources mentioned in the report for guidance in optical setup.

Experience using MATLAB and Mathematica is beneficial, as many of the programs here used for simulation and processing were written in these software packages.

Experience using CUDA for GPU parallel processing is highly recommended, if not necessary for continuing the work on the real-time holographic display. Additionally, familiarity with GPU rendering techniques is necessary for real time generation.

Optical Setup

Figures 4 and 5 below detail the current optical setup.

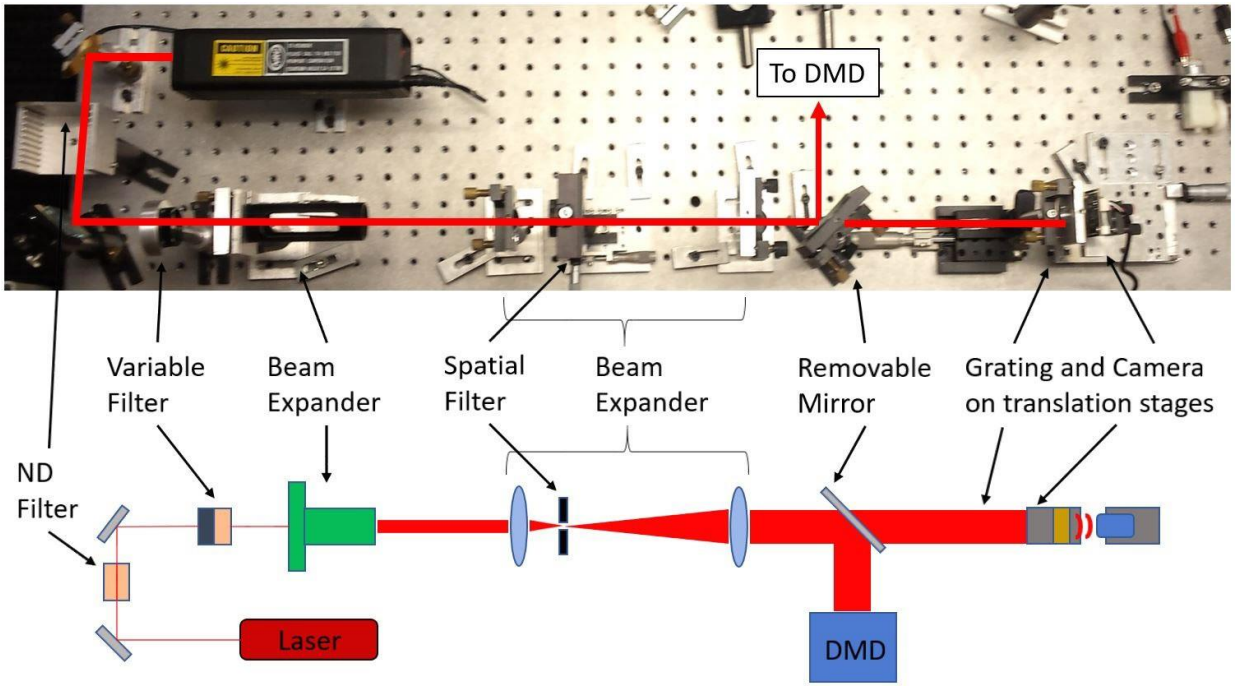

Figure 4: Optical Setup

The setup features a Helium Neon laser (HeNe), steering mirrors, 2 beam expanders, two kinds of filtering, a spatial filter which cleans up the beam, the near field diffraction grating and camera mounted on stages, as well as the DMD and an insertable mirror for steering to the DMD. Details about some of the items in the setup can be found in Appendix D. For understanding this setup, Thorlabs and Edmund Optics have great guides on things like basic laser alignment, spatial filtering, and lense selection:

https://www.edmundoptics.com/resources/application-notes/lasers/simplifying-laser-alignment/

https://www.thorlabs.com/newgrouppage9.cfm?objectgroup_id=10768

Another useful resource for execution of setup, including alignment through lenses is the lab manual for the Physical Optics and Fourier Transforms senior lab. The first few pages which have the most relevant details pertaining to setup can be found in *Repository* > *Books and Other Sources* > *Physica Opticas and Fourier Transform Lab Manual* . The lab manual also contains material relevant to understanding the Fourier filtering used in processing grating images for making the Talbot carpet, which is discussed later in this paper.

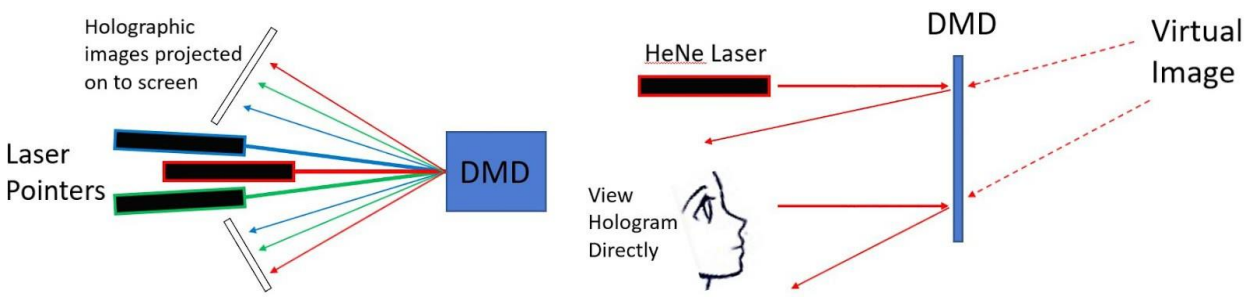

Figure 5: Displaying Holograms

Holograms can be viewed directly when shining the HeNe on the DMD at near-normal incidence. Finding the best angle for viewing them can be tricky, and there are many repeated images of the hologram projected at different angles with different amounts of power due to the "blazed grating" properties of the DMD. To view one, find the holographic projections using a card. Pick one with sufficient brightness to be viewed easily, and place an iris in line with it, such that, when looking through the iris from opposite the DMD, you are looking along the line of the projection, as in Figure 6 below.

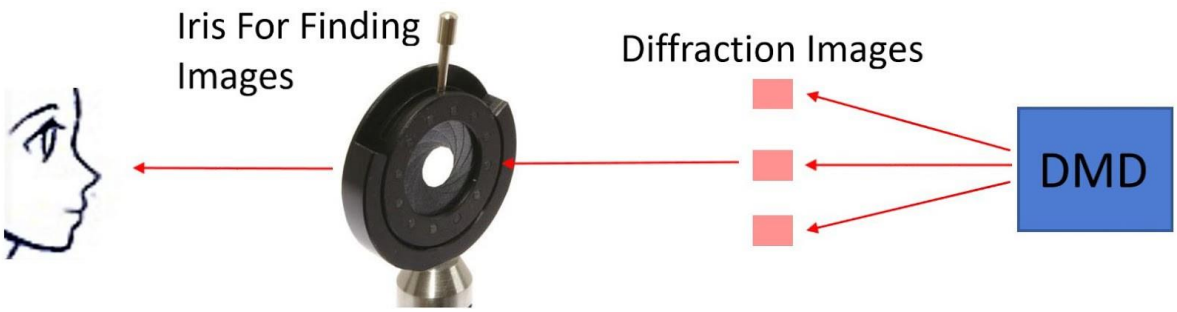

Figure 6: Finding virtual images for direct viewing

The DMD produces two images (There are 2 solutions to the wave equation evolving from the array of micromirrors). The image that is produced in front of the DMD is called the "real image" and can be projected onto a card. The image that appears behind the DMD is called the "virtual image", because it can be seen behind the DMD, but no card can be put there for the image to be projected onto. Though the images can be projected onto a screen at any distance, they come into focus at a specific distance, which should be very close to the distance between the object and the screen used for the calculation of the holographic diffraction pattern. This shows that they are *actual holograms* rather than just reflected laser.

Near Field Work

Single Slit Fresnel Diffraction

As a first step, we used the 100 µm slit provided with the available optics to capture images of the near field diffraction patterns to compare to simulations written prior to experimentation. Figure 7 contains the results from both the experimentally captured images, and that obtained via simulation using Huygen's Principle of light propagation. The fourth row of the figure details the relative and absolute distances between the experimental and simulated diffraction patterns. The initial discrepancy is mostly caused by the thickness of the glass cover over the pixels on the camera.

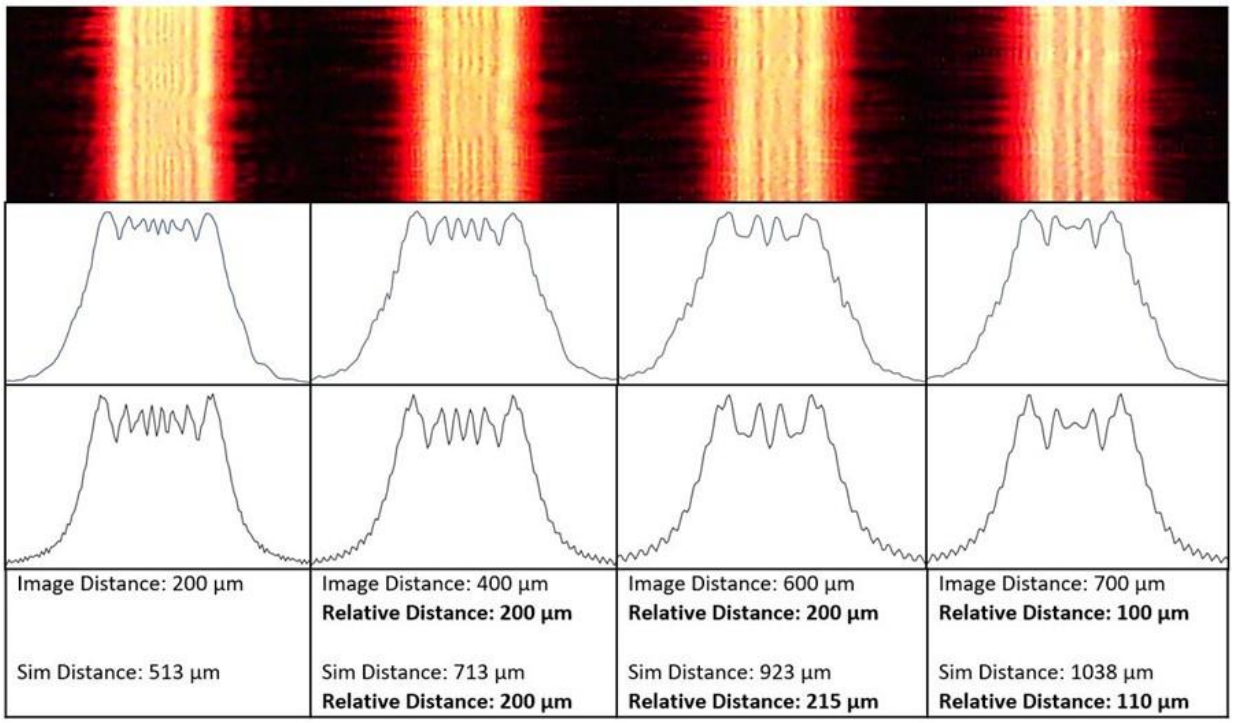

Figure 7: Diffraction patterns from 100 µm slit: (top) Experimental Images, (middle) Experimental intensity plot, (bottom) Simulation using Huygens principle

Comparing the experimental intensity plots to the simulation, there are a few notable differences. The first is in the height of the peaks and valleys of the diffraction pattern. The simulations shown here actually display the square root of the simulated intensity. (It is easier to see the similarities this way since the peaks and valleys in the experimental intensity plot are squished.) Furthermore, close inspection shows that there is a slight bulge in the base the experimental intensity plots as seen in the figure below.

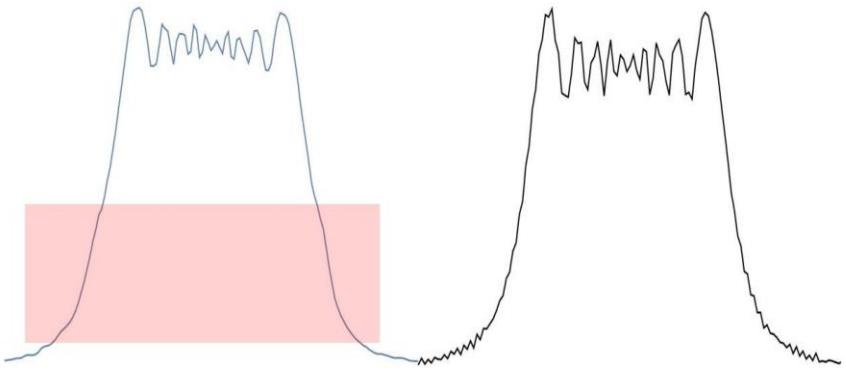

Figure 8: (left) Bulge in experimental intensity plot highlighted. (right) Square root of the simulated intensity. Arbitrary units.

It turns out that these effects are produced by the RGB filter layer on the camera pixels. Figure 9 below shows an image of the separated image layers. The maxing-out of the red pixels basically amounts to the diffraction patterns in the blue and green pixels being set atop a pedestal in the intensity plot above. For a discussion of ways to deal with these effects see the *Potential Next Steps*: *Near Field: Camera* section of the paper.

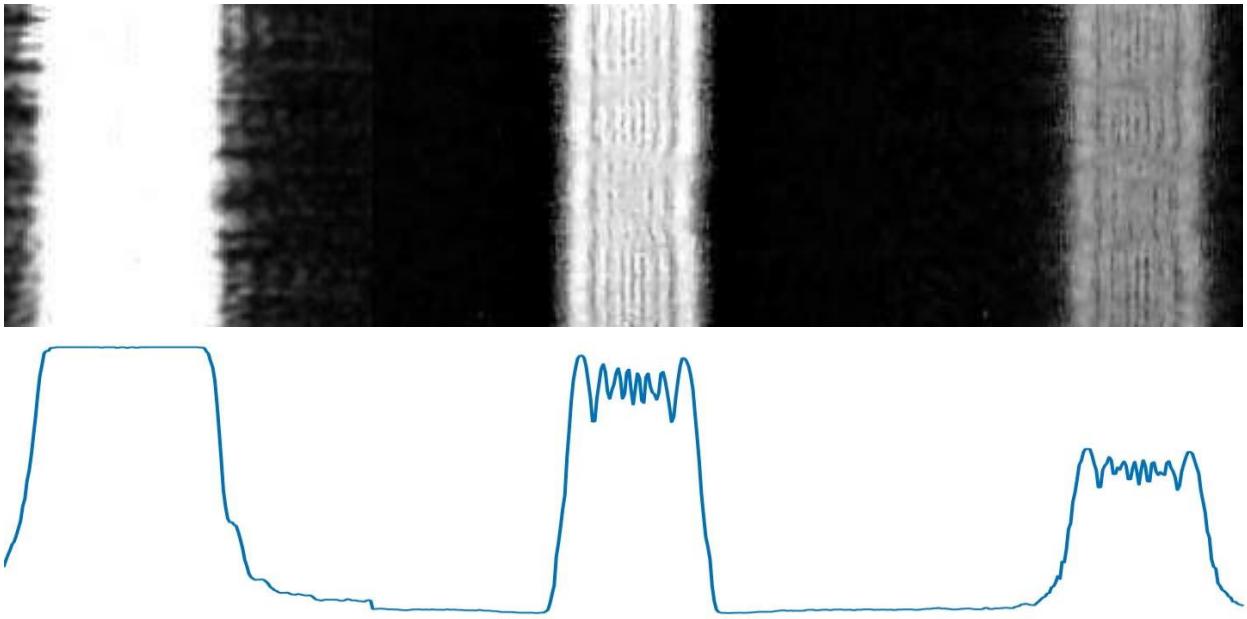

Figure 9: RGB pixel values as returned by the camera.

Images acquired from our more-precise, fabricated slits yielded similar results in terms of fringe compression and distance offset, but introduced a new phenomenon. Displayed below, in Figure 10, is the image taken using our fabricated slit where it can be observed the candy cane striping across entire image. Our hypothesis behind this behavior was the interference due to the transmitted light through the sides of the slit. As a possible solution, we increased the thickness of deposited metal, but ultimately could not image any diffraction patterns due to time constraints.

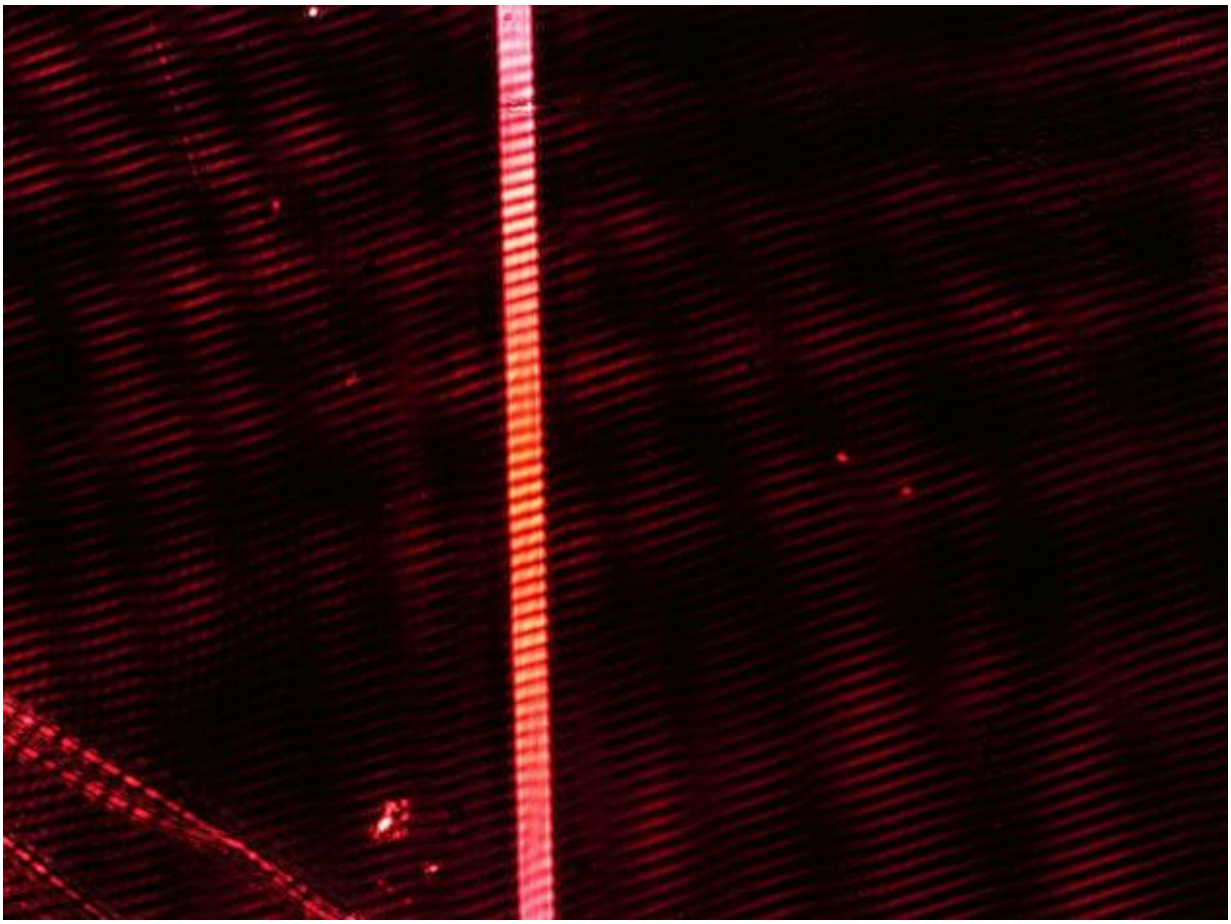

Figure 10: Laser incident on 100 µm slit. Note the light penetrating through the metal and the striped frequency seen across the entire image.

Talbot Carpet

Prior to beginning work on the Talbot Carpet, a simulation was conducted to probe the relationship between slit width, slit center-center spacing (c-t-c), and overall resolution of the Talbot Carpet image. Our simulation was written incorporating the size and spacing of pixels on our CCD. Figure 11 contains the results from the simulation of two different configurations.

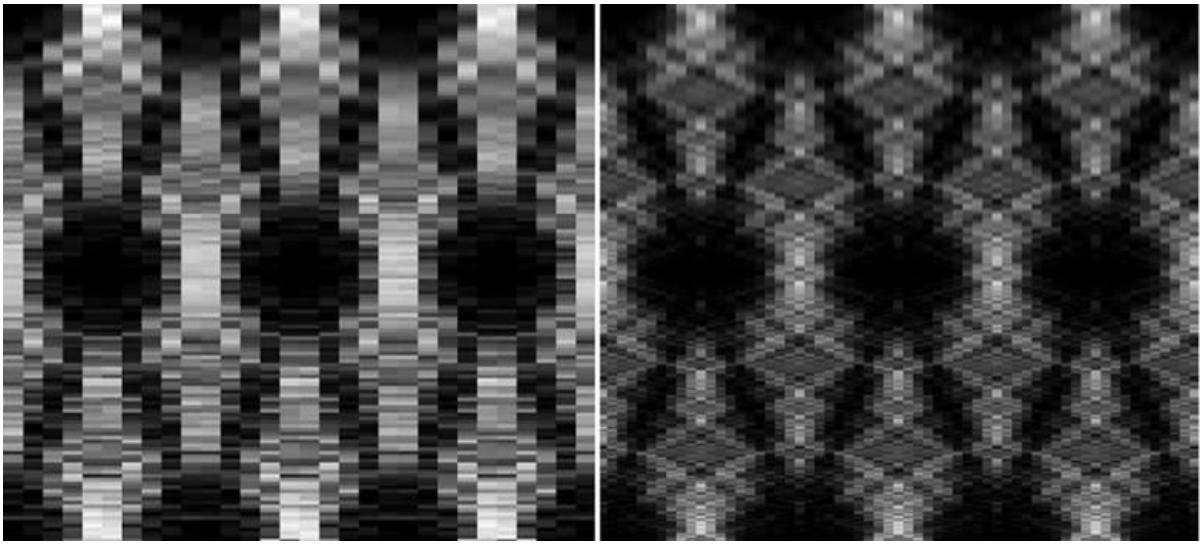

Figure 11: Simulated Talbot carpets used to determine optimal configuration for grating fabrication. Grating dimensions: (left) 5 µm width, 15 µm spacing, 0.7 mm Talbot length, (right) 10 µm width, 30 µm spacing, 2.8 mm Talbot length.

Note the increase in resolution and increase in Talbot length from doubling the width and c-t-c spacing parameters. Given the reasonable talbot length and higher resolution, we decided to choose the configuration with widths of 10 *µm*, and c-t-c spacing of 30 *µm*. Using the specified parameters, a grating was fabricated followed by subsequent image acquisition. Details about the fabricated grating and image results can be found in the following sections.

Grating Fabrication

For fabrication, a photolithographic process was used. Mainly, a clean glass slide was first cleaved into the proper size. Shipley 1805 photoresist was then spin coated onto the glass substrate and baked. Following baking, the substrate was exposed through the use of a Heidelberg Instruments uPG501 maskless aligner, defining the desired pattern into the resist. Next, the exposed resist was developed with MF-319 developer, and the sample was loaded into the Lab-18 physical vapour deposition (PVD) system. Once loaded, ~600 [A] of aluminum was deposited onto the sample. Finally, following deposition, portions of the metal layer and the remaining resist were lifted off through the use of Remover PG. Specifically, the samples were allowed to soak for ~30 min in solution before being placed into a sonicator for 5-15 minutes of sonication. The complete samples were then rinsed in deionized (DI) water and dried with nitrogen. An illustration of the fabrication process in given in figure 12 below, while a detailed, step by step, description is given in the fabrication instructions document located in the repository.

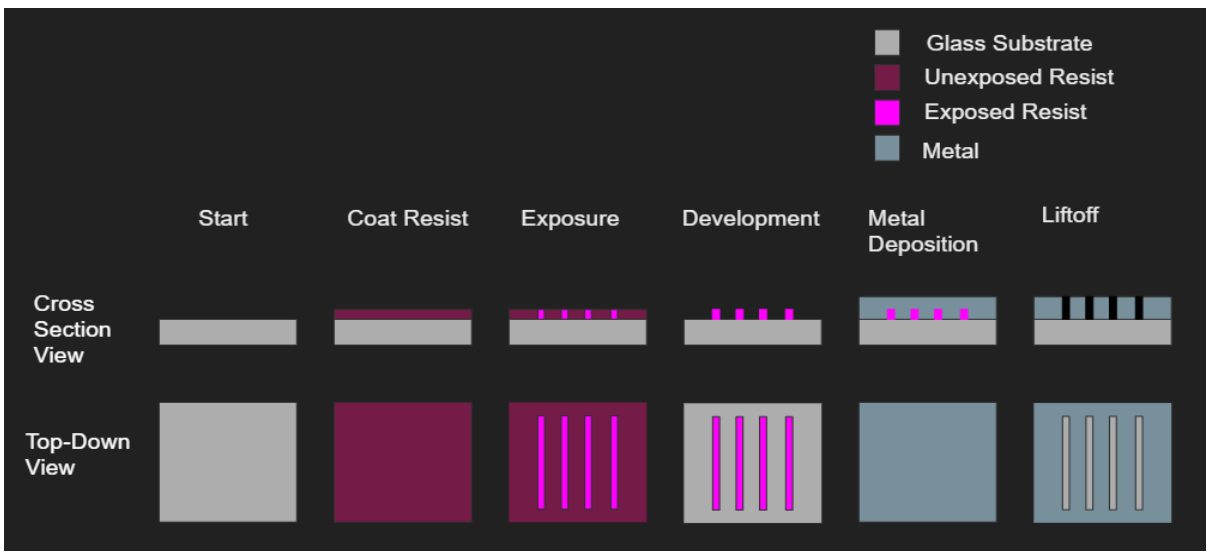

Figure 12: Fabrication process.

During fabrication, several problems were encountered, most of which pertaining to the metalization and liftoff. For metallization, initial attempts were made using chromium (Cr) of thickness varying from 200 [A] to 1000 [A]. It was found that in order to block a sufficient amount of light, a Cr layer of at least 1000 [A] would be needed; however, such a thick layer would not produce a reliable liftoff. To combat this, the metalization material was changed to aluminum, as the aluminum allowed for less optical transmission per unit thickness when compared to Cr. With this change, the necessary metal thickness was reduced to approximately 600 [A]. An optical transmission calculator is linked in the appendix. Despite this, reliable liftoff was still not possible at this thickness, so an additional sonication step was implemented. With this new liftoff process, the sample was soaked in the Remover PG for ~30

min, and then sonicated for 5 to 15 minutes, checking for complete lift off every few minutes. The additional agitation afforded by the sonication allowed the films to properly lift-off; however, the agitation also increased the occurrence of over liftoff and unwanted breaks in the metal film. This issue had limited effects on the smaller 1x1 [in^2] grating; however, the larger 2x2 [in^2] were significantly impacted by the issue. This can be clearly seen in the result section below. To remedy this issue, it is recommended to adopt the use of lift off resist (LOR). In this process, a two resist stack of LOR and S1805 is spun onto the substrate. The sample is then exposed and developed as usual, but the developer undercuts the LOR. This allows the resist to act as a shadow mask, and results in a cleaner lift off by creating a natural break in the metal layer after deposition. This is illustrated in Figure 13 below:

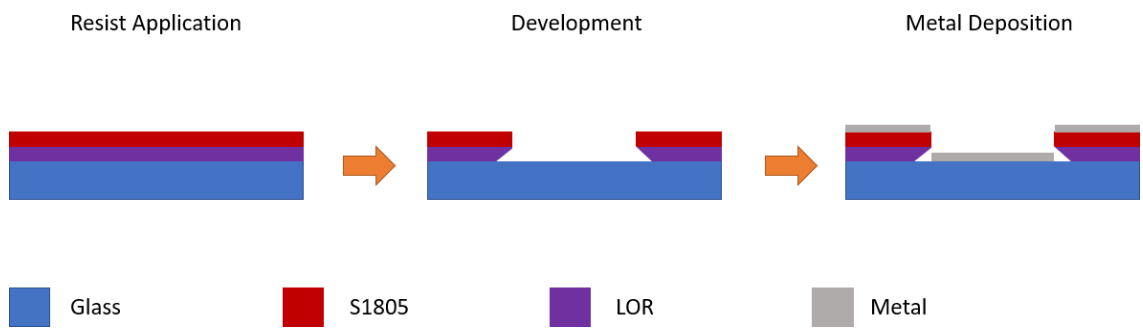

Figure 13: Lithography with LOR

Beyond this, cleaner patterns could also be achieved by replacing the liftoff based process to an etch based process. In this process, metal would be deposited onto the substrate first, followed by resist application and photolithography. The resist would then act as a stencil or mask, and the uncovered metal would be etched away with either a chemical etch, or plasma based etch (such as RIE). The resist mask would then be stripped revealing the finished grating.

Using this process, several components were fabricated. These included a 100 [um] single slit, a 1x1 [in^2] diffraction grating with a 10 [um] slit width and 30 [um] spacing, and a 2x2 [in^2] diffraction grating with a 30 [um] slit width and 90 [um] spacing.

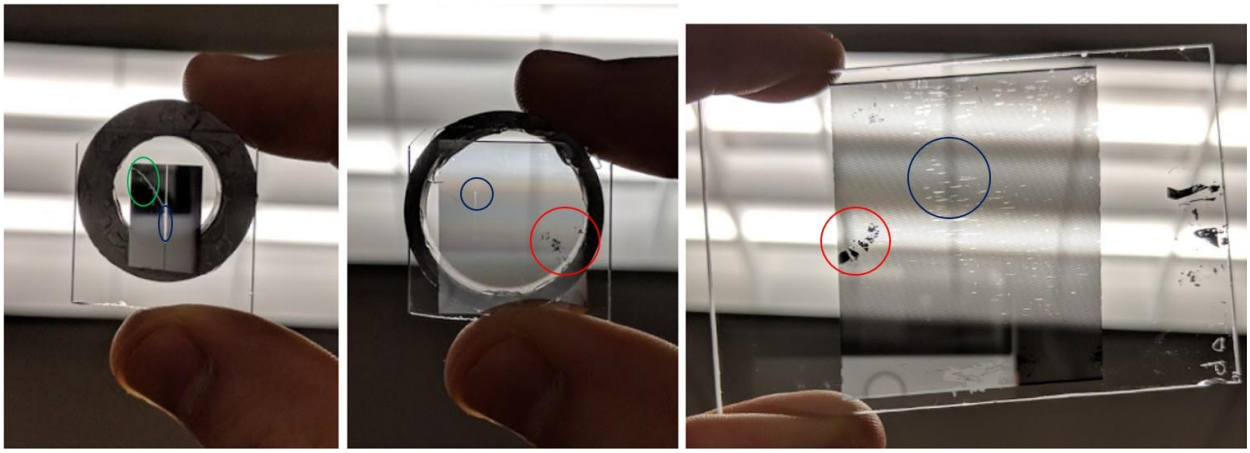

Figure 14: Images of the three samples:

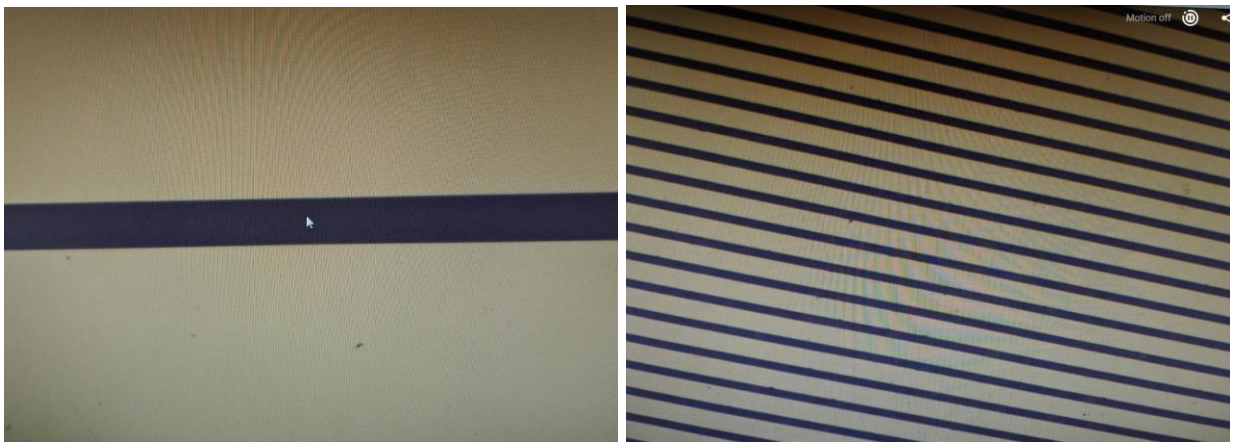

Figure 15: Optical micrographs of (left) single slit and (right) 1x1 grating:

As can be seen from the above figures, defects were present in all three samples; however, defects were sufficiently localized on the single slit and 1x1 grating, allowing them to be used for experiment. The 2x2 grating had a much higher defect count, thus the process changes detailed in the problems section above are strongly recommended. Specific defects that were observed include scratches (green circle), incomplete liftoff (red circles), and over liftoff (blue circles).

Talbot Carpet Image Processing

Figure 16 below displays and example of the images taken which where put together to make the Talbot carpet.

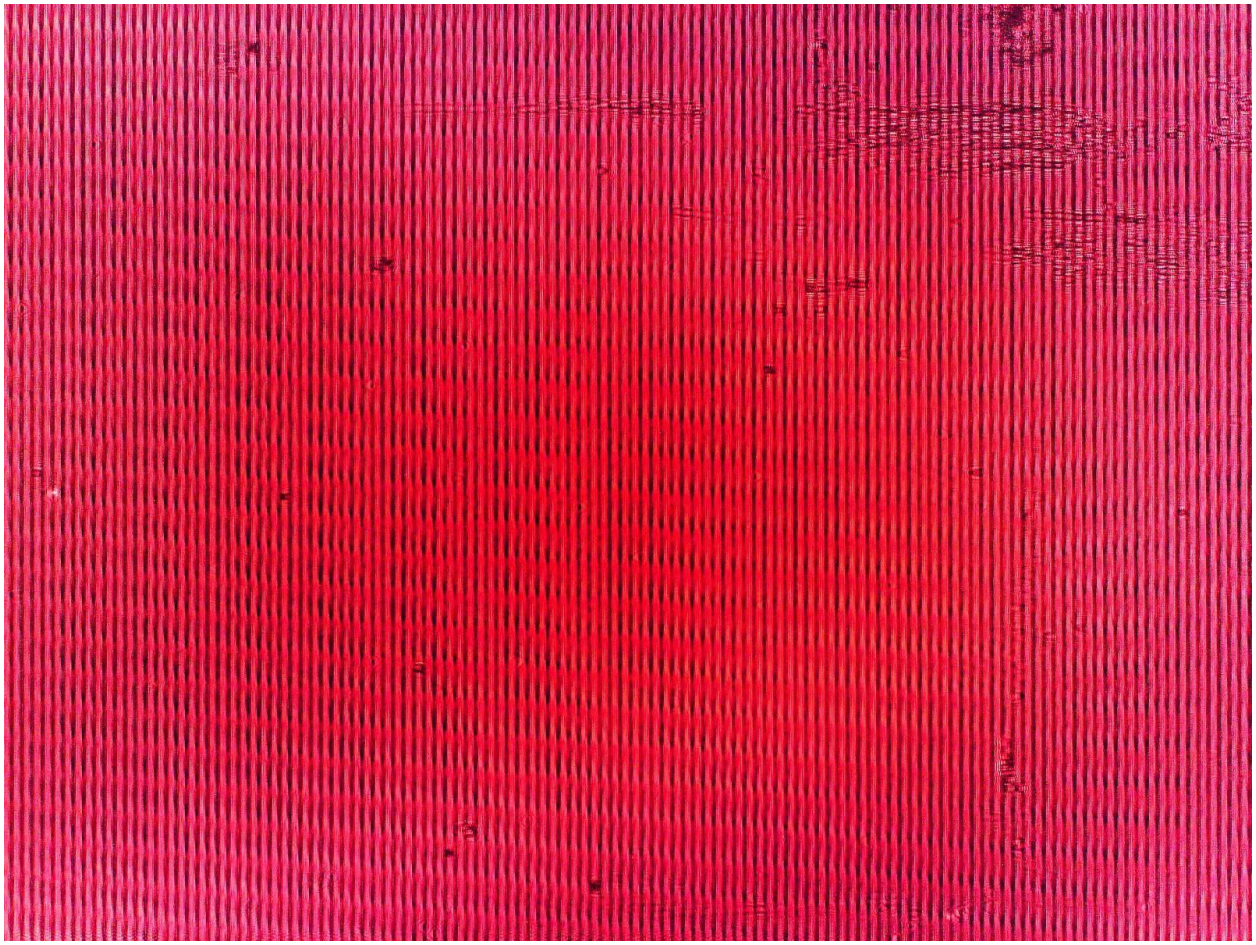

Figure 16: Image of 10 micron width, 30 micron center-to-center grating at about 10 micron distance.

The image above has some notable defects. First of all, there appear to be scratches and dings on the camera glass. More interestingly, there is a close-to horizontal frequency creating almost horizontal lines across the image. The grating slits are almost exactly vertically aligned with the camera pixels and produce the fast frequencies in the horizontal direction. The source of the the vertical frequencies is currently unknown, and might be worth investigating. Note that the gaussian shape of the laser intensity is also apparent in the image.

In order to clean up the images, filtering was performed in the frequency space of the images using fast Fourier transforms in MATLAB. The images and MATLAB functions that perform this filtering as well as assemble the images into the Talbot carpet can be found in *Repository > Talbot Carpet > Talbot Image Processing Results*. Since the entire image shown is not

necessary for creating a Talbot carpet, the images were cropped before filtering was applied. Figure 17 is an example of a cropped and filtered image.

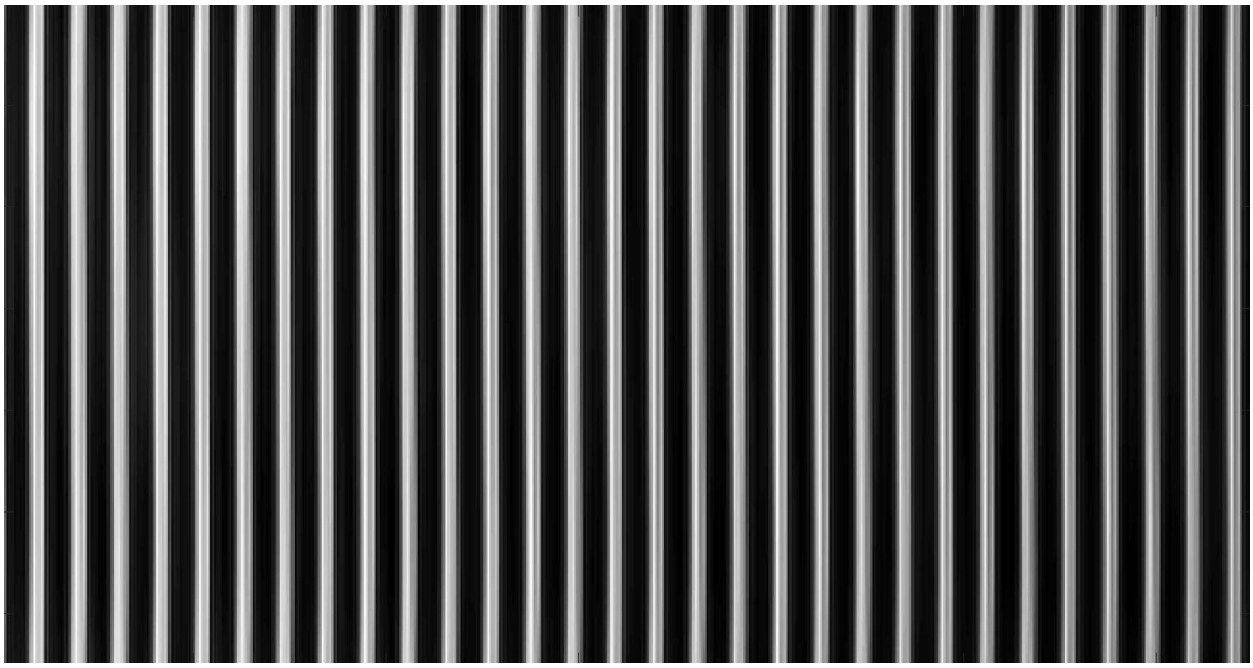

Figure 17: Cropped and filtered image of 10 micron slit, 30 micron center-to-center spacing grating at about 10 micron distance.

More details and intermediate images showing the Fourier filtering can be found in the repository. If you are not familiar with Fourier filtering, the Physical Optics and Fourier Transforms lab manual contains good information. The programs here literally do the exact same thing as the physical optics in that lab. Taking a line out from every filtered image and putting them together produces the following result.

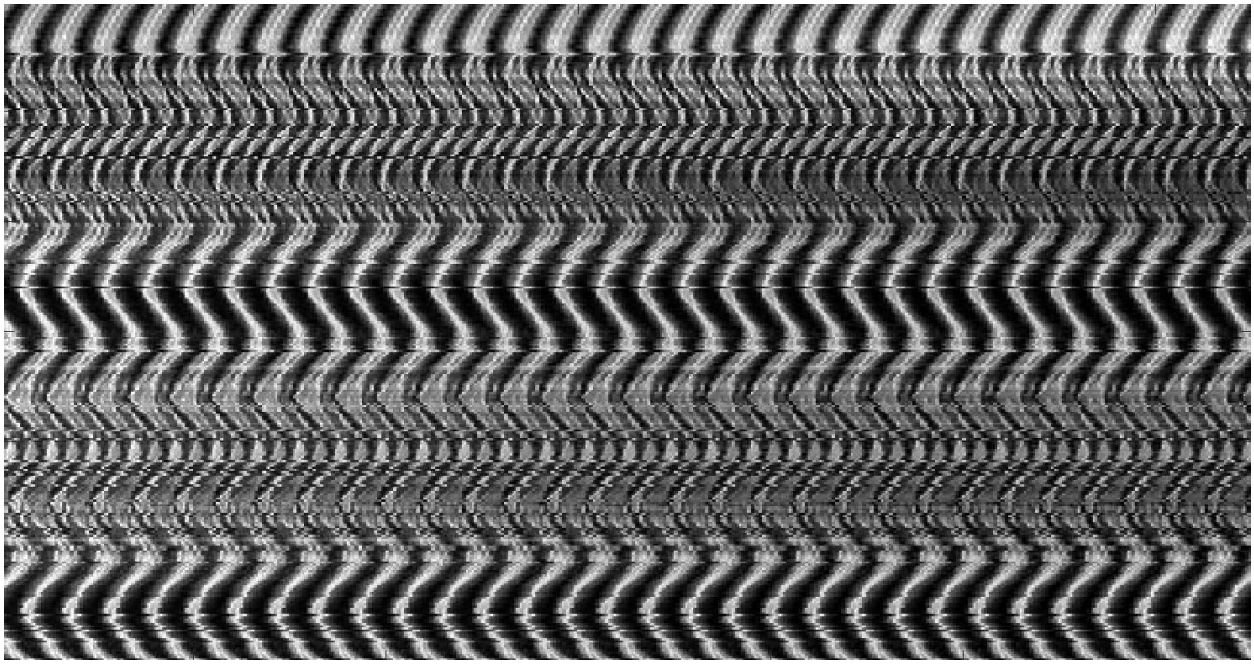

Figure 18: Raw Talbot carpet

The Talbot carpet above was made using approximately 300 images spaced 10 *µm* apart up to a total distance of approximately 3 *mm* from the grating. As you can see, there are very wavy lines in this image, presumably due to slop in the stage used to translate the camera. The sharp discontinuities in the images are due to backlash when driving the screw “backwards” after overshooting the desired image distance. Lineout at the top of the image all look the same because the initially the camera and slit are touching, and as you back off, the system first lets out some flex while the camera and grating are still essentially touching. Some suggestions for fixing these problems can be found in the *Potential Next Steps* section of this report.

To programatically fix the alignment issues seen above, a script was made that compares each line-out with the next adjacent line-out. Details on the program can be found in the MATLAB files in *Repository* > *Talbot Carpet* > *Talbot Images and MATLAB Processing Scripts.zip*. Figure 19 shows the resulting image.

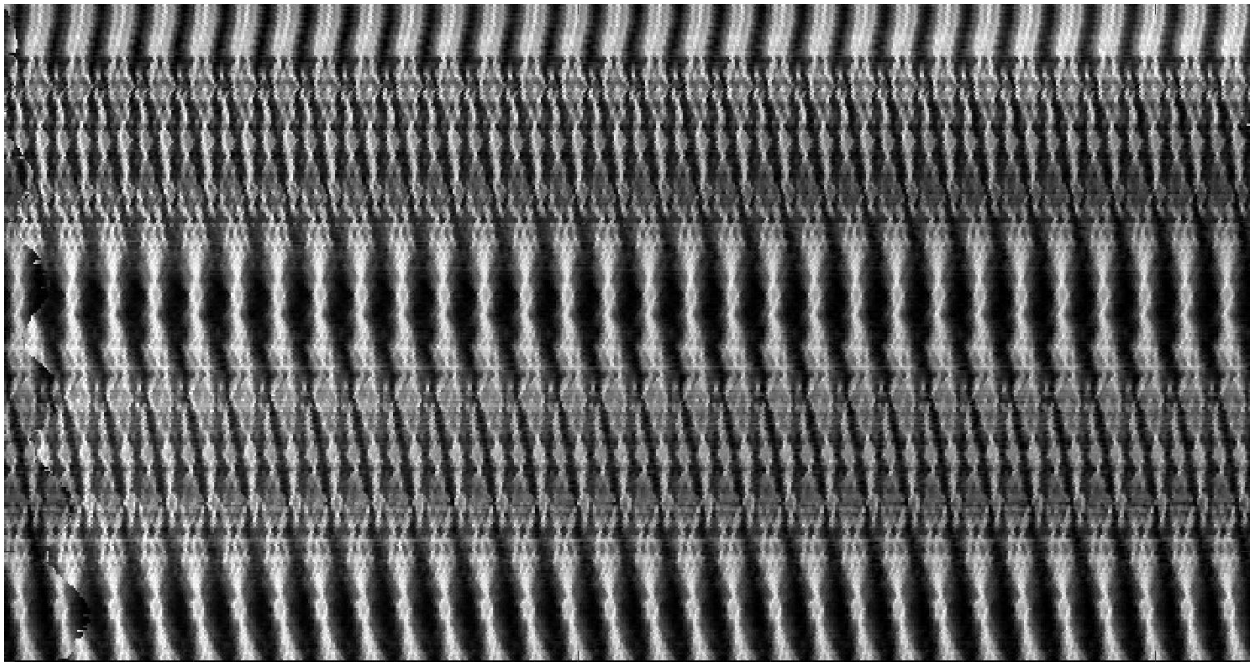

Figure 19: Shifted Talbot carpet image

As you can see, the program does not work perfectly. It gets some of the shifting wrong. However, it also doesn't do too poorly and the Talbot carpet is easily seen. Figure 20 below compares a cropped section of the above image with simulation.

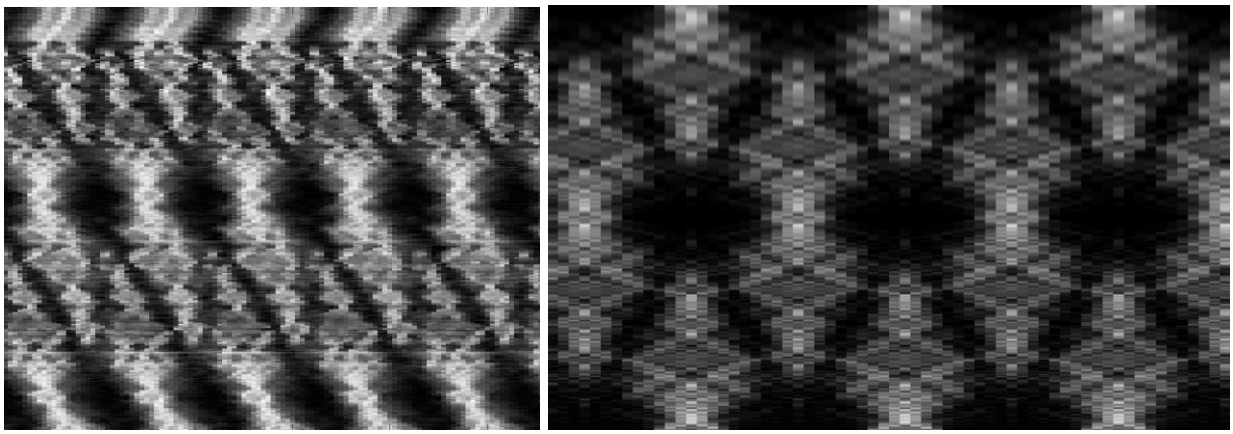

Figure 20: (left) experimental Talbot carpet, (right) simulated Talbot carpet

The patterns are strikingly similar, and an even closer inspection reveals quite nice qualitative agreement. Though exact comparisons of Talbot length have not yet been done (we acquired these results very very late in the semester, it is indeed the case that the experimental Talbot length is actually quite close to the predicted length. That is, the predicted length is 2.8 mm, and the experimental images were taken out to a distance of 3 mm reading on the screw, before which is clearly seen the repetition of the grating pattern.

Holography

Introduction to Holography

Holograms are generally made by mixing a monochromatic reference wave of known phase periodicity in with monochromatic light from an object on the recording medium (holographic plate). The mixing of the phases of the 2 beams modulates the amplitude of the light recorded and therefore stores the phase information. When the object is no longer present, but reference beam is shone through the holographic plate, it diffracts in such a way as to recreate the light from the original object, phase and all. For more detailed information see the *Holography Handout* in *Repository* > *Books and Other Sources*.

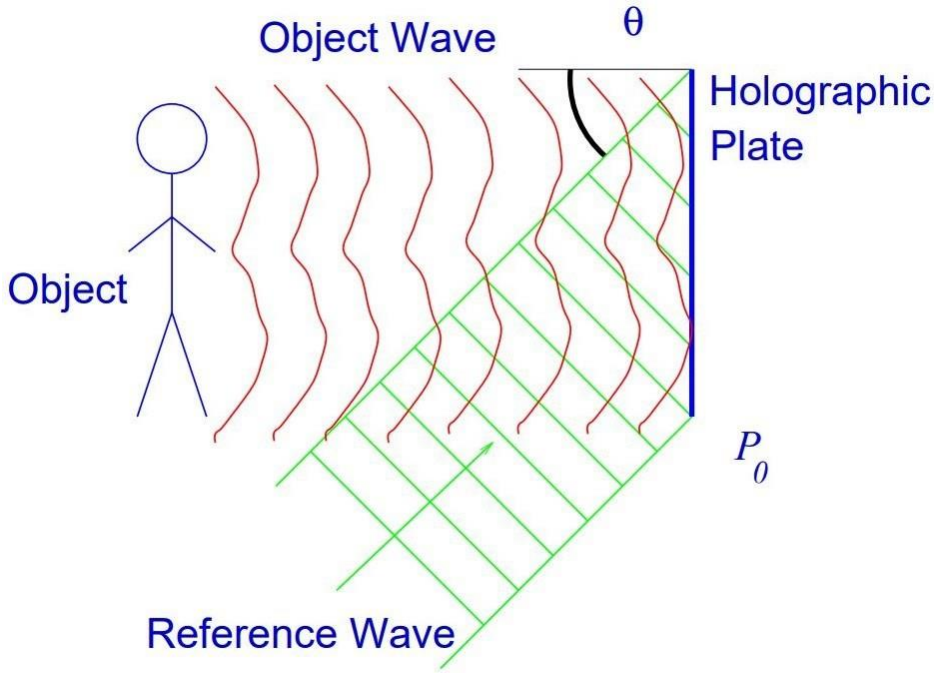

Figure 21: Mixing of Object and Reference wave on the holographic plate.

For this project, there is no actual object present, and a diffraction pattern is not being recorded. Rather, the diffraction pattern is being calculated from an object represented by an array of points. This is what is then displayed on the DMD and used to make the hologram. Two ways of calculating the pattern were attempted. The “Ray tracing” or “Huygens principle” method was used and run on GPUs with great success. The drawbacks to this method generally have to do with computation time, but hardware advances have made this trivial. The Fresnel transform method was attempted using MATLAB and generally runs pretty quickly, however there is some mistake in the program which is causing errors in the produced patterns.

Holographic Diffraction Patterns on GPU

We were able to demonstrate real time holography, and presented several demos at the Physics picnic: spinning rings, n-sided polygons, and a cube. These holograms were produced with interference patterns that produced locally on an NVidia GTX 780 graphics card (a.k.a. GPU). The simulation performed what is essentially a large Huygens principle simulation using a GPU programming library created by NVidia called CUDA,. For a point on the DMD, the contribution from each point source in the simulation was found and added to the contribution from the reference beam as shown in Equation 1 where Ref(x,y) is the reference beam contribution and Pn a point source contribution on the DMD.

$$V(x,\ y)\ =\left|Ref(x,\ y)\ +\Sigma P_n\ (x,\ y)\right| \tag{1}$$

Several simplifications were made to increase the performance of the simulation. The reference beam was assumed to lie in the plane of the table, and to have a small angle Θ relative to the normal of the DMD. It was also assumed that the DMD lie in the XY plane with a corner at the origin. This gave rise to the following:

$$Ref(x,y) = \ cos\ (\ \tfrac{2\pi x}{\lambda}\ sin(\theta)\ ) \tag{2}$$

$$R_n(x,\ y)\ =\sqrt{(p_x-x)^2\ +\ (p_y-y)^2+\ p_z{}^2} \tag{3}$$

$$P_n(x,\ y)\ = p_w\ *\ R_n(x,\ y)^{-1/2}\ +\ sin\ \left(Remainder\left(\tfrac{R_n(x,\ y)}{\lambda}\right) * \tfrac{2\pi}{\lambda}\ \right) \tag{4}$$

Calculating these intensity values across the entire DMD results in a $V(x,\ y)$ array describing the interference pattern. Initially this pattern was turned into a bitmap image of the appropriate resolution for the DMD; however it is necessary to scale the image brightness. Our first prototype would find the 95th percentile value and set that to be full brightness (white). The remaining values were linearly scaled accordingly, resulting in a grayscale bitmap which could be saved. This bitmap was then displayed on the DMD as if it were just an external display. Once the reference beam was projected onto the DMD, the hologram was visible. These first holograms were basic and consisted only of a few points.

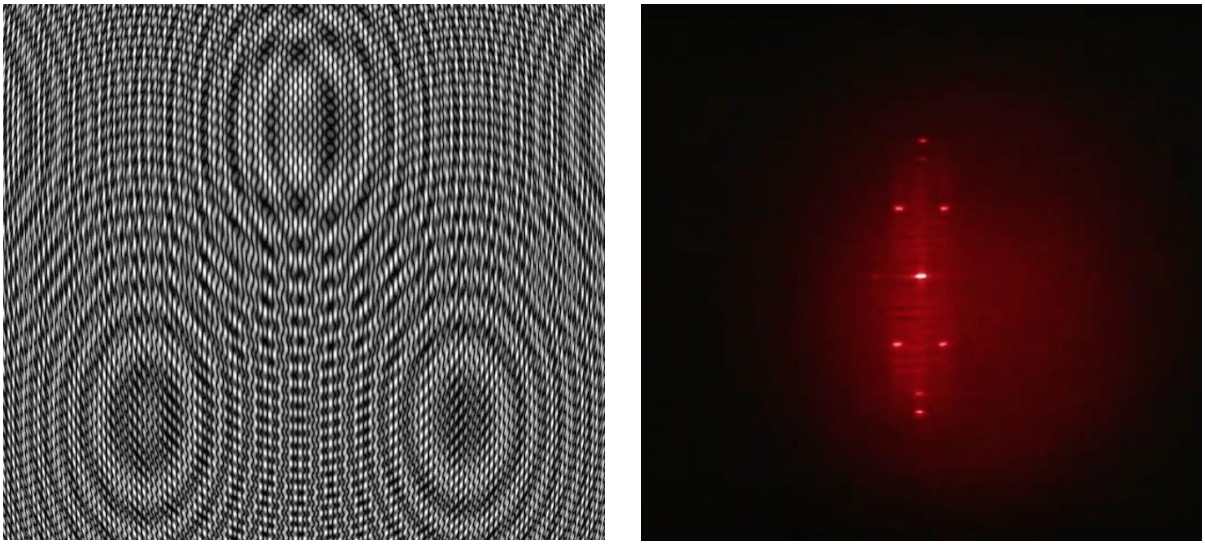

Figure 22: One of our first holograms, three points in a triangle. (left) the interference pattern (right) is the reconstructed hologram as viewed by a camera

Producing the still holograms required some trial and error. Our first few attempts produced holograms that had severe aliasing issues. To fix this we rendered the images further from the DMD surface until the aliasing was no longer an issue. At 30 cm from the DMD the aliasing was drastically reduced. At 60 cm the aliasing is completely gone, but the extra distance made viewing the holograms much more difficult. Because of this we used 30 cm for our holograms.

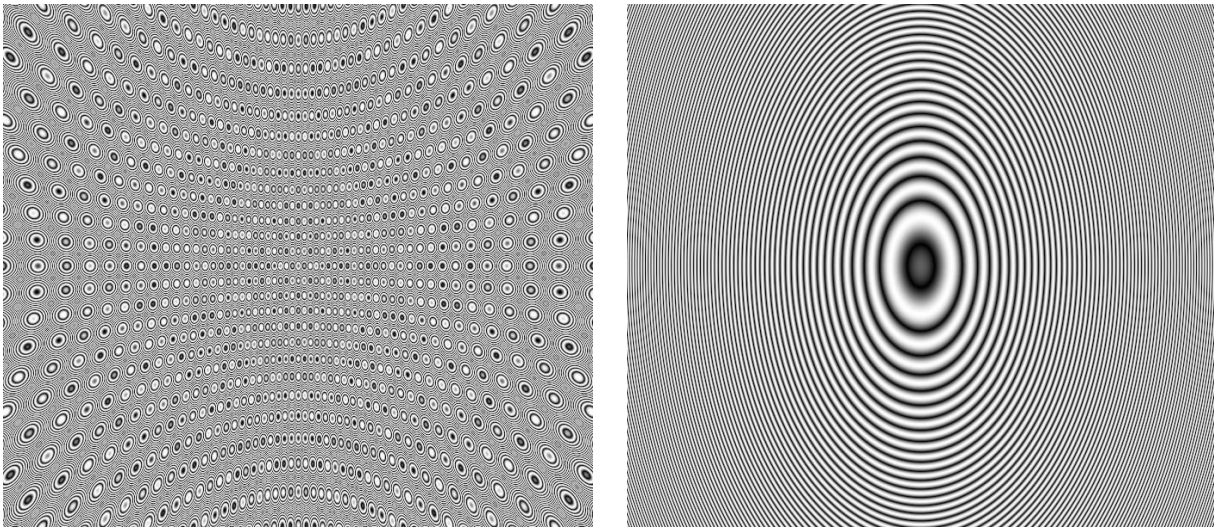

Figure 23: (left) Badly aliased diffraction pattern rendered at 1 cm from the DMD surface, (right) The same diffraction pattern rendered at 30cm from DMD surface shows drastically reduced aliasing.

After this proof-of-concept work was done, we worked to generate and display these patterns in real time. Rendering the pattern immediately, as opposed to writing a bitmap, is very fast. The pattern is already on the graphics card after simulation, so drawing it to the screen can be done very quickly. Some work was done with OpenGL and GLFW to quickly get the pattern on screen. Ultimately we were able to have the pattern appear in a preview on a computer monitor while also being rendered directly to the DMD. For our demonstration at the Physics picnic we wrote a few interactive demos which would generate some different holograms: perturbed rings, N-sided polygons, and a spinning cube. These holograms are visible in two way, either by looking at the DMD and focusing at the proper distance behind it, or by throwing the image onto a screen. For our demonstrations we displayed these images on a screen for two reasons. First, the holograms are difficult to find when looking directly at the DMD, and even when one is able to locate the image, the eye often refuses to focus on an arbitrary distance. Additionally, looking directly into the DMD poses the safety risk of laser-eye contact, and blinded spectators are notoriously unhappy spectators. We primarily used the HeNe laser used for the near field experiments to create the holograms, but other lasers can also be used; however because the simulation is for 632 *nm* light, smaller and larger wavelength lasers will produce images at a different scale. Further, unfiltered (spatially noisy) laser light can be used for viewing athe the cost of some noise in the reproduction. In fact for our presentation, we used red, green, and blue laser pens, and while the images were somewhat noisy, the results were spectacularly bright. This extra intensity was important for showing the holograms at the Physics Picnic due to the excess light in the atrium.

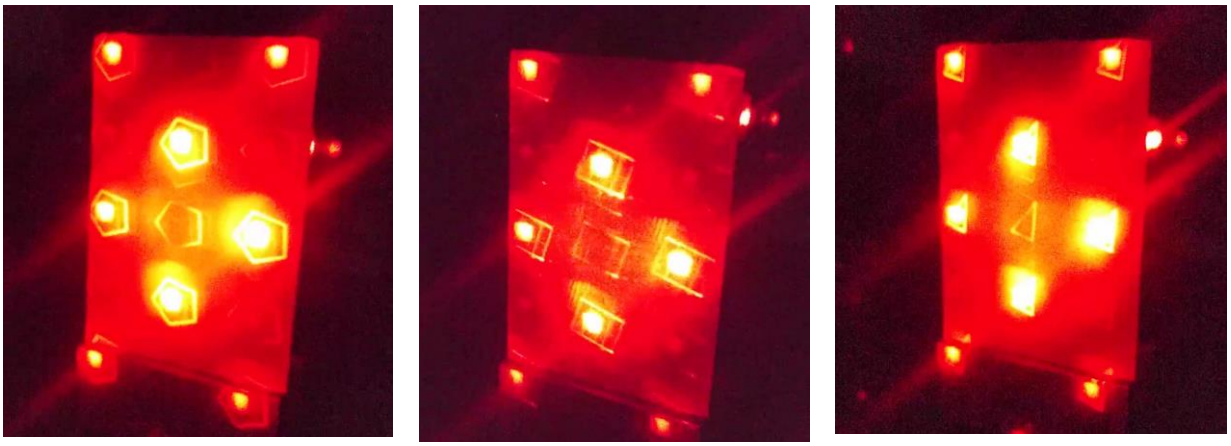

Figure 24: Holograms as generated for demonstration. From left to right: pentagons, cubes, and triangles. Note that the central image in each hologram does not contain the bright spot characteristic of the other images.

We aimed to produce a holographic movie using the point cloud data from an Xbox Kinect. The kinect is promising “holographic camera” because it is designed to measure distance and track motion. The kinect provides both a color image and a depth map which can be used to calculate

a point cloud in real time. Each frame from the kinect could thus be used to generate a point cloud which would be the input for a hologram computation. There are a few issues with the kinect. One, the official SDK is no longer officially supported. The opensource community has developed an alternative through the OpenKinect project. The api provided, freenect, was used in our exploration of the kinect, however documentation is sparse. Additionally, first generation kinect sensors have a relatively low depth image resolution. This isn't so much of an issue as much as it is a limitation towards making crisp holograms. A newer version of the kinect is available and four times the depth resolution, but is rather expensive to purchase. A higher resolution image could prove to be more headache than it's worth for local real time holography, as the amount of computational work would be greater. We could solve this by performing the hologram calculations through a distributed network such as at the Ohio Supercomputer Center.

Holographic Diffraction in MATLAB: Fresnel Transform

Holographic diffraction images can be created using a numerical approach to Fresnel's integral for diffraction. The program created used MATLAB examples from *"Numerical Simulation of Optical Wave Propagation, With examples in MATLAB"* [3]to implement the numerical integration as a fresnel modulated fast fourier transform. An overview of the function files created, and their purpose, is found in Appendix B. The program takes the input image of which a hologram is desired, parameters from the optical setup and the discretization of the DMD. The program then uses the numerical fourier transform to calculate the diffraction pattern on a plane from the image. The calculation is done by shifting to an intermediate plane before another transformation to the final image plane. Further details can be found in the program comments and in the numerical simulations text [3]. The diffraction pattern is added to a reference beam, then saved as a bitmap image of the required resolution. The matlab program can be found in the Repository folder along with all of the images created. Two examples are presented here, one of a test figure and the second from a flipbook. Figure 25 shows a single slit test image used to verify the program and the resulting diffraction pattern image. This diffraction pattern image did not produces a viewable hologram when displayed on the DMD and illuminated with the reference laser.

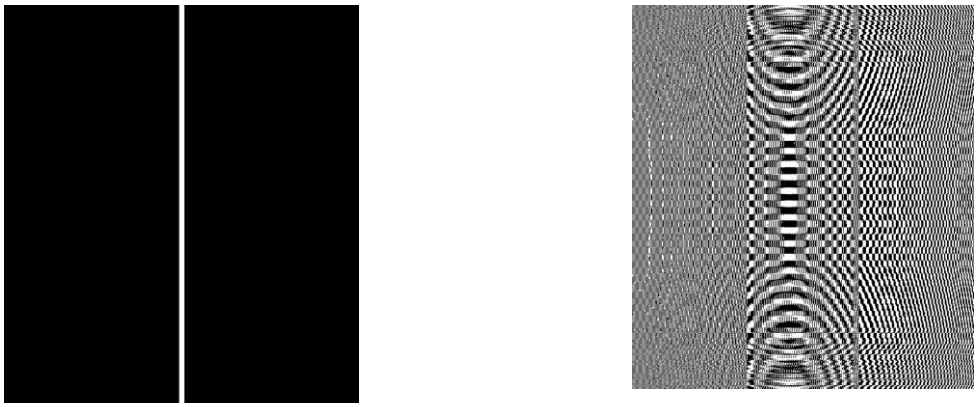

Figure 25: (left) 3 pixel wide (~32 um on DMD) single slit source image. (right) Hologram diffraction pattern calculated from (a) using the matlab program.

To produce a holographic movie using this program, a flipbook was utilized. Single frames were transformed into holographic diffraction images and scanned through to produce a movie effect. An example of one frame and its holographic diffraction pattern is shown in Figure 26. These diffraction patterns failed to produce a holographic image when displayed on the DMD.

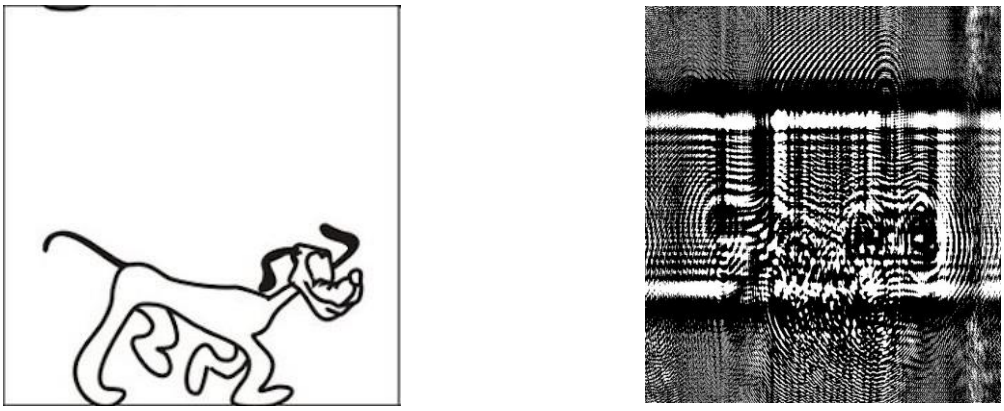

Figure 26: (left) Image number 19 from the walking dog flip book. (right) Hologram diffraction pattern calculated using the MATLAB program.

The issues with this program are thought to stem from how MATLAB computes fast fourier transforms and the spatial frequencies shifting function used. There are multiple numerical methods MATLAB utilizes for fast fourier transform, some of which may propagate errors that destroy the diffraction image. The different types of numerical methods could be investigated for accuracy. When computing fast fourier transforms, MATLAB expects the low spatial frequency terms to be around the edges of the 2D matrix, which is inverted from a physical optics spatial frequency plane. There is a fast fourier shift function in matlab that moves the high spatial frequency terms to the edges and brings the low frequency terms to the center. Numerical calculations of a physical optics system require this. The machinery used to shift these points is unknown, which could be another source of error in the calculation.

Potential Next Steps

Near Field

Camera

There are some issues with the images captured by the camera. The camera was chosen because of its small pixel size (1.4 micron). Which allows for resolution of small diffraction pattern changes. However, there are two issues with this camera for our purposes. First, as with all color cameras, there is a “Bayer filter” over the pixels to capture color intensity. This means that the true resolution is lower than advertised because only one third of the pixels are capturing red light. Second, the camera outputs a JPEG format that applies compression to the image. This compression is somewhat of a black box to us, so it is difficult to be sure how this affects the images.

Possible solutions to this include purchasing a different camera, one that reports “mono” or black and white images. Unfortunately, we could not find any ready-to-go cameras of this sort with 1.4 micron resolution. However, the DMM 24UJ003-ML USB 3.0 monochrome board camera manufactured by The Image Source might be a suitable option.

The final option is to purchase a camera with the desired resolution and an RGB filter that can output the raw pixel signal. Then by adjusting the intensity of the laser, measuring it accurately with a diode, and recording the response from the camera, a response curve could be created for each pixel. This curve could then be applied to diffraction images to recreate the actual on-pixel intensity.

Absolute Image Distance

One issue with the images taken is with the initial distance between the camera and the diffraction grating. If the camera is put close enough to be in contact with the grating, there is still some unknown distance between the grating and the pixels. This discrepancy is apparent when comparing experimental images to simulation. To help correct for this, a measurement of the thickness of the glass cover that is glued onto the front of the camera pixels should be made. This measurement can be made by looking at the camera under a microscope, focusing on the front surface of the glass and then focusing on the pixels (or filter layer) behind the glass and noting the distance. Another method to find the cover thickness is by ellipsometry. There should be a usable ellipsometer in a cleanroom on campus.

Talbot Images

The raw talbot images show a couple discrepancies. First, there is a “waviness” in the image, possibly caused by alignment shifts in the stage as it is moved. As well, there a couple sharp discontinuities in alignment. These are most likely caused by backlash in the screw as distances were “overshot” when turning the screw one way, and the screw was turned back.

For further Talbot carpets, a more accurate stage should be used: one that has less “slop” and backlash. As well, it would be best to motorize the stage and take automated continuous images rather than move the stage and capture images one at a time by hand. This would make Talbot carpet attempts more streamline.

Other further work includes taking images with the larger grating that was manufactured. Despite its imperfections, it may be that it can produce good images. Other, better gratings could also be manufactured.

Laser Characterization

Finally, it would generally be good to have some characterization of the intensity of the light apparent on the gratings and DMD. This can be achieved through measurements using a power meter made from a diode and circuit, the pieces to which are already available in the lab.

Holography

When the laser is incident on the DMD surface, the DMD acts as a blazed diffraction grating. The diffraction grating creates many replica images all around the DMD. The properties of the blazed grating effects could be understood in greater detail, as it may be important moving forward. In each of the images created from the holographic diffraction and blazed grating diffraction, there is a bright center dot that washes out the hologram. The bright center dot has been noted in other works regarding holography with DMDs. When the hologram is displayed as a real image onto a notecard or piece of paper, there is a hologram in the center of the blazed grating diffraction images that has little to no background light and does not contain the bright center dot. A video of the center hologram is located in the Repository. A tube or aperture that blocks the blazed grating diffraction images for the viewing of only the center hologram could be implemented.

Kinect

We didn’t finish our work on getting the kinect point cloud information into the CUDA simulation, however we came very close. We can retrieve the depth buffer in realtime, transform this buffer into a point cloud and send it to the graphics card. Our work can be found in the *kinect* and *glfw_cuda* branches of the github repository. This raw point cloud data is too large to render in real time (640 * 480 points) so several reductions are needed. First, filtering out the background,

by ignoring points with a depth greater than roughly 2.5 meters can eliminate about two thirds of the points. If the goal is to record a movie, and play it back it is enough to store this data for several seconds worth of recording and process it offline. The NVidia GTX 780 GPU we used can process about 13,000 points per second, which means it can do about 400 points per frame at 30 frames per second. The exciting challenge is to do the kinect processing in real time. A few approaches worth investigating include rending only a random sampling of points per frame - that is choose around 400 points from the point cloud data and compute the hologram with those only. Obviously, this won't look as good as a simulation using all of the points, but at 30 frame per second it may look good enough. Additionally, our simulation allows for variable brightness of points, this allows from interesting explorations into texturing holograms. One source of texture information (variable value and intensity info) available on the kinect is the stream from its RGB camera. The brightness of points in the simulation could be related to the red value from this texture or even just the brightness of a pixel. With red, green, and blue reference beams, it would even be possible to do true color holography. Another, less obvious source of texture can be derived from the depth image itself. Ambient occlusion is a technique used in computer graphics to give the impression of surface texture in lighting calculations. Namely, a technique known as screen space ambient occlusion (SSAO) computes the gradient on a depth image, and relates surface roughness to brightness. A SSAO calculation could be quickly calculated and could yield results that look a bit more three-dimensional.

Cuda

The CUDA simulation is currently highly optimized for a GTX 780 GPU. Work distribution parameters (grid size, block size, and thread count) are tuned for the GTX 780 specifically. That is not to say that simulation would run poorly on another piece of hardware; however for ideal performance tuning should be done. The process of tuning is a mix of intuition and hardware knowledge. Nvidia provides a piece of software such as the Nvidia Visual Profiler as part of their CUDA toolkit, these are invaluable for identifying bottlenecks. Some research was done on the performance of different GPUs, namely the NVidia P100 (available at OSC) and the NVidia GTX 1080 (a high end consumer card as of the time of writing). Both of these graphics cards perform similarly, handling about 35,000 points per second. Some investigation could be done into distributing the work across multiple GPUs as well, either on the same system via SLI, or across a network using OpenMPI. As an additional note, a few clever optimizations can be done to the simulation, but would require a bit of a trial and error approach. Most generally, the simulation uses a few slower operations in its per point contribution calculation. These could be replaced with faster, though less accurate, approximations. Specifically Taylor expansions of sine and cosine.

MATLAB Hologram Calculations

Due to the output diffraction images, It is believed that there is a bug in the program. However, before digging in to the MATLAB program, it is necessary to get a solid understanding of what it

is supposed to do. That is, to understand how to derive the expression for Fresnel diffraction and recognize the part that is a Fourier transform. Then, the primary function to investigate is *two_step_holo.m,* and the *ft2.m* and *ift2.m* functions embedded within. Getting a full understanding of MATLAB's built in functions *fftshift(), ifftshift(), fft2(), and ifft2()* is highly recommended, if not absolutely necessary. Finally, a full understanding of all the distance calculations is necessary, and the calculations in the program should be double checked for accuracy. Descriptions of these can be found in Appendix B.

Beyond this, one of the primary things that can be adjusted is the relative strength of the reference wave to the object wave, since they are added together, and one can wash-out the other.

A final suggestion is to play with the field over which the hologram is calculated. The function is "one-to-one". That is, if your object image contains X many pixels, so will your holographic pattern. Padding the image with zeros, and then cropping the extra pixels off the diffraction pattern for display on the DMD might yield good results.

Appendix

Appendix A: Lightcrafter EVM and DMD(DLP 3000)

Interfacing

There are three ways to easily connect with the lightcrafter EVM (LC) for uploading images. The first method, and the one ultimately used, was connection by HDMI. To connect to the LC by HDMI, power on the LC and wait for the start-up sequence to finish. Connect the HDMI cable to the LC and a personal computer. There is an input mode selection button on the LC that is used to select the desired input type. The device will act as an external display for Windows 10 and Linux/GNU operating systems natively. If using macOS, a display manipulation software is required, such as SwitchResX. The refresh rate for this connection type is 60Hz. The second method to upload images involves an microSD memory card. As this limited to 1 image per second, it is not advised for holographic movies. The images desired to upload require a naming scheme as follows: "*00_imagename.bmp", "01_imagename.bmp", "02_imagename.bmp".* The images require a numeral and underscore before the file name, where the numeral length and the file name is the same. These images can be loaded onto the microSD card with a configuration text file. The file name should be "config.txt" and include the number of seconds between the display of each image on line 1, and the sequence of file names, one per line, thereafter. The final method is uploading a single static image via the Lightcraver EVM GUI software. This software can be found on the Lightcrafter EVM Texas Instruments webpage. The GUI has a lot of functionality, most of which would not work. The GUI can be used to successfully upload a single static image, and be used to change the input mode. Greater detail can be found in the manuals for the Lightcrafter EVM and DLP3000 (DMD chip).

Specifications

The DLP3000 DMD chip used on the LC has mirror size of 10.8 *um* corner to next corner and 7.637 *um* edge to next edge. Figure A1 show small portion of the DMD array with dimensions and orientation. Only the red colored mirrors count as a column, whereas alternating red and blue mirrors count as rows. The orientation of the mirrors lends to a single mirror not representing a single pixel from an uploaded image. The consequence of this requires thought about discretization when creating the hologram interferograms. The native resolution of the DMD chip is 608 x 684 (Width x Height). Other output resolutions are available, with a full list in the Lightcrafter EVM manual.

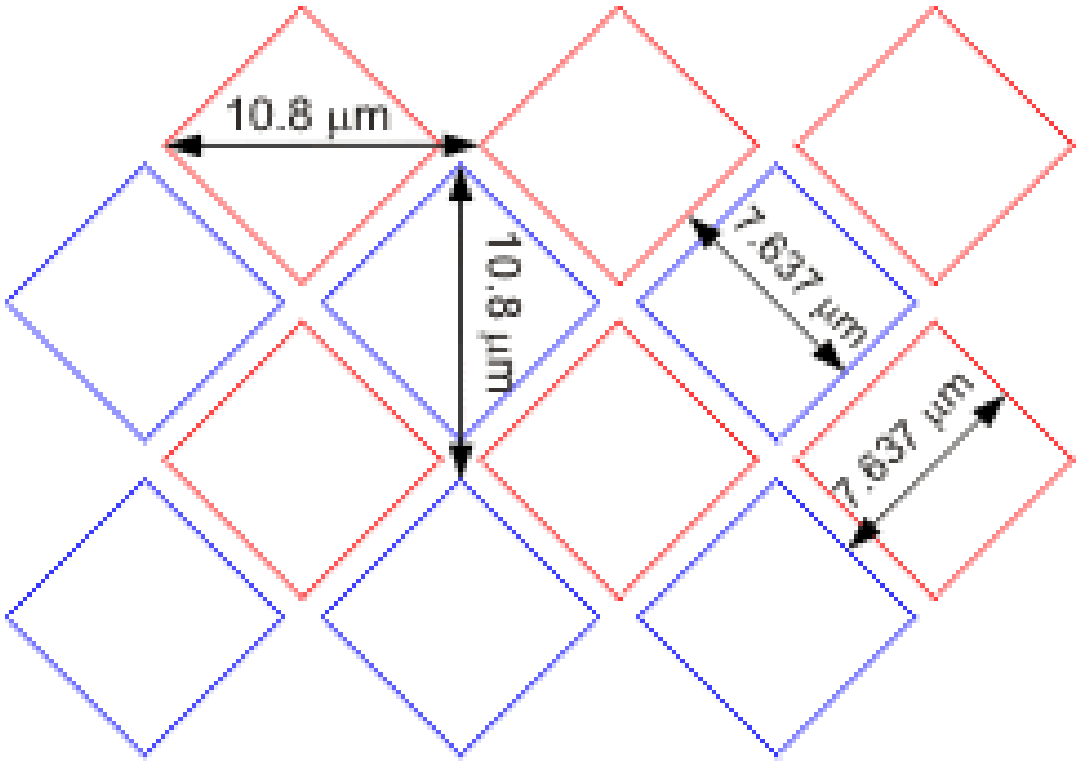

Figure A1: Small section of DMD mirror array with dimensions and mirror orientation.

The refresh rate of the DMD can be as large as 4000 Hz. The method found to implement a 4000 *Hz* refresh rate required the use of onboard memory. The onboard memory is small, limiting the length of holographic movie. A way to seek images loaded on a microSD card was not found, however, this can be investigated further. The HDMI connection is limited to 60 *Hz*, which is quick enough for a half plane or quarter plane per frame. Meaning, one interferogram could be calculated over four frames to create a larger hologram at the cost of framerate.

Usable Inputs

A bitmap image of resolution 608 x 684 is required when using the GUI or the microSD card. When using the HDMI input, the image type is not restrictive. Something to consider, however, is how the DMD implements greyscale. The DMD flickers the mirrors, using pulse-width-modulating, to decrease the time dependant intensity at a pixel. From this consideration, black and white, single bit width, bitmap images were created for any input mode.

Appendix B: Matlab Holographic Calculation

Function Files

main.m

This matlab file implements the required functions to produce a hologram diffraction pattern image. The input parameters are listed here, along with saving the final image.

two_step_holo.m

This function file implements a two step fresnel transformation of the source image to create a diffraction pattern. The source image can be of any type and resolution, which is processed in *main.m*

hologramField.m

This function file adds the reference beam component to the diffraction pattern created by *two_step_holo.m*. The output of this function is a matrix containing the intensities at each pixel for the hologram.

binaryImage.m

This function file creates a binary image of the hologram matrix calculated in *hologramField.m.* The function utilizes a built in matlab function that selects what pixels will be white or black.

discreteSize.m

This function file calculates the discretization step sizes for the source image plane, the intermediate diffraction plane and the final diffraction plane. These step sizes are required for an accurate holographic diffraction pattern image by numerical methods.

f2t.m

This function file replaces the 2D FFT built in matlab function. Since matlab places the low spatial frequency terms around the edges of the matrix, they have to be shifted to the center before calculation. The fresnel transform expects low spatial frequencies near the origin, and not at the boundaries.

Supplementary Files

DMDimages.m

This function is an outline for creating source images for testing. Any number of pixels can be set to white for quick tests. The images are saved as single bit width images of resolution 608 x 684.

Appendix C: CUDA and OpenGL

The repository for the CUDA holography code : https://github.com/LarryTheKing/SuperHolo
Libraries/APIs/technologies used:

CUDA:

Nvidia's general GPU computing api and framework. Freely available along with several dev tools in the cuda toolkit. Runs on windows, linux, macOS, and at the OSC. GPU vendor specific. Used in this project to do the bulk of the simulation computation.

OpenGL:

An open source graphics api used to render 2D and 3D graphics across many platforms.
Used in this project to render the final diffraction patterns in real time.

GLFW:

A cross platform api for creating OpenGL contexts and handling system window creating and input processing.
Used for window generation and input handling in our demos.

Microsoft Kinect:

A gaming peripheral released for the Xbox360, later updated for the XboxOne. Uses a conventional RGB camera paired with a LIDAR to capture color and depth information.

OpenKinect/LibFreenect:

The open source community's kinect project. Provides access to the depth and color cameras and provides control over some other aspects of the kinect.